

Trabalho Prático

Heaps no Algoritmo de Dijkstra

Disciplina: Análise de Algoritmos

Objetivo

O objetivo deste trabalho é investigar como a escolha da fila de prioridade afeta a análise assintótica e o desempenho observado do algoritmo de Dijkstra. Os alunos deverão implementar heaps binário, binomial e de Fibonacci sob uma interface comum e utilizar cada um deles sem alterar a lógica central do algoritmo.

Modelo da Atividade

O trabalho poderá ser realizado individualmente ou em grupos de, no máximo, três pessoas. A entrega deverá preservar a estrutura e os nomes de arquivos definidos neste enunciado.

Premissas

- Linguagem: Python 3.11 ou superior;
- Os grafos são direcionados e representados por listas de adjacência;
- Todo peso é um número real finito e não negativo;
- Os vértices são inteiros consecutivos de 0 a $n - 1$;
- A análise deve considerar $n = |V|$ e $m = |E|$.

O uso de pesos negativos deve ser rejeitado com `ValueError`. Vértices inalcançáveis devem receber distância `math.inf` e predecessor `None`.

Parte 1: Filas de Prioridade (3,0 pontos)

Complete o arquivo `aluno/heaps.py`. Cada classe deve oferecer a seguinte interface pública:

```
push(vertex, priority) -> handle
decrease_key(handle, new_priority) -> None
pop_min() -> tuple[vertex, priority]
__len__() -> int
```

As classes exigidas são `BinaryHeap`, `BinomialHeap` e `FibonacciHeap`. O método `push` devolve um identificador opaco (*handle*), posteriormente aceito por `decrease_key`. As prioridades não aumentam; uma tentativa de aumento deve lançar `ValueError`. O método `pop_min`, quando chamado em uma estrutura vazia, deve lançar `IndexError`.

Não é permitido usar `heapq` nem outra implementação pronta de fila de prioridade nas três classes. Estruturas auxiliares comuns da linguagem são permitidas.

Parte 2: Algoritmo de Dijkstra (2,5 pontos)

Complete o arquivo `aluno/dijkstra.py`. A função

```
dijkstra(graph, source, heap_class) -> tuple[dist, pred]
```

deve instanciar `heap_class` e usar exclusivamente sua interface pública. Não duplique o algoritmo de Dijkstra para cada heap. Em caso de empate, qualquer árvore de caminhos mínimos válida será aceita.

Parte 3: Análise Assintótica (2,5 pontos)

Preencha `aluno/respostas.json`, usando somente as categorias indicadas no modelo. Use o pior caso para os heaps binário e binomial e os limites amortizados para o heap de Fibonacci, conforme a tabela comparativa usual.

Além das operações individuais, informe a complexidade do algoritmo de Dijkstra com listas de adjacência e `decrease_key`, sem supor que o grafo é conexo.

Parte 4: Experimento e Discussão (2,0 pontos)

Complete `aluno/benchmark.py` e produza `aluno/relatorio.md`, com no máximo 1.200 palavras. Compare as três estruturas em pelo menos três famílias de grafos e em pelo menos quatro tamanhos por família. Use as mesmas instâncias e fontes em cada comparação, faça aquecimento, execute repetições e apresente uma medida de dispersão.

O relatório deve:

1. descrever a máquina, a versão do Python, o gerador, a semente e o protocolo;
2. apresentar tabelas ou gráficos com unidades e dispersão;
3. relacionar os resultados às operações dominantes de cada família de grafo;
4. explicar por que melhor complexidade assintótica pode não significar menor tempo para os tamanhos testados;
5. registrar limitações e ameaças à validade.

Entrega

A entrega deverá conter os seis arquivos abaixo, preservando seus nomes e assinaturas:

- `autores.json`;
- `heaps.py`;
- `dijkstra.py`;
- `respostas.json`;
- `benchmark.py`;
- `relatorio.md`.

O arquivo `autores.json` deve conter de um a três autores. Informe o nome completo e o RA de cada integrante exatamente como constam no sistema acadêmico. O RA é um identificador textual: zeros à esquerda devem ser preservados. Autores sem nome ou RA, RAs repetidos no grupo e mais de três integrantes tornam a identificação inválida.

O código deve ser autoral, legível e acompanhado de referências para qualquer material externo utilizado.

Os seis arquivos devem ser colocados diretamente na raiz de um único arquivo compactado no formato `.zip`, sem pastas intermediárias. O nome do arquivo deve seguir o padrão `Trabalho_Heap_RA.zip`, utilizando o RA do primeiro autor informado em `autores.json`.

Critérios de Avaliação

- Implementação e contratos dos três heaps: 3,0 pontos;
- Correção do algoritmo de Dijkstra e tratamento de casos-limite: 2,5 pontos;
- Análise assintótica: 2,5 pontos;
- Experimento e discussão: 2,0 pontos.

Os primeiros 8,0 pontos serão avaliados automaticamente. O relatório experimental será avaliado conforme a rubrica divulgada pelo professor.

Observações

Os experimentos devem ser executados em ambiente controlado. Utilize a mesma instância de grafo e a mesma fonte ao comparar os três heaps. Registre a semente do gerador, realize aquecimento, repita as medições e apresente uma medida de dispersão. Decisões metodológicas e limitações devem ser justificadas no relatório.